Level 8 · about 40 h · Python 3.8+

Project: Two tiny coroutines gathered into one result
Examples run on CPython 3.12.3. **You will be able to:**

- Write an async def and run it with asyncio.run
- Wait for two tasks with asyncio.gather
- Tell a coroutine object from the value it returns
- Name the docs page for tasks and timeouts

Coroutines and Tasks

asyncio.run drives one coroutine

`async def` defines a coroutine. Calling it does not run the body; it returns a coroutine object. `asyncio.run` creates an event loop, runs that coroutine, and closes the loop.

```
import asyncio

async def ping():
    return "pong"

print(asyncio.run(ping()))
```

```
Output (CPython 3.12.3):
pong
```

gather waits for more than one

`asyncio.gather` schedules the awaitables together and returns their results in order. The tasks page in the docs covers `gather`, cancellation, and timeouts.

```
import asyncio

async def add(a, b):
    return a + b

async def main():
    return await asyncio.gather(add(1, 2), add(3, 4))
```

```
print(asyncio.run(main()))
```

```
Output (CPython 3.12.3):
[3, 7]
```

A coroutine is not the result

Printing the coroutine object without awaiting it does not print the return value. This sample closes it so the build does not warn about a coroutine that was never awaited.

```
async def ping():
    return "pong"

coro = ping()
print(type(coro).__name__)
coro.close()
```

```
Output (CPython 3.12.3):
coroutine
```

Check yourself

1. What does calling an `async def` return before you await it?
2. What does `asyncio.gather` preserve about order?
3. Which docs page describes `create_task` and timeouts?

Sources

- Python docs: asyncio
- Python docs: Coroutines and Tasks

Code © CypherMonkeyLabs, MIT. Text CC BY-NC-SA 4.0. Educational material, not accredited coursework; no outcome or income promised. Built 2026-10-18.